A D PATEL INSTITUTE OF TECHNOLOGY
DEPARTMENT OF INFORMATION TECHNOLOGY

TO-DO LIST APPLICATION

USING PYTHON AND TKINTER

(TaskMate)

A Mini Project Report

A report submitted

In partial fulfilment of the requirements for the Degree of

Bachelor of Technology

In

Information Technology

Semester – V

Subject:

Python Programming

By

Megh Chetankumar Patel

(12402080601081)

Faculty Guide

Keyur Patel

Assistant Professor

Academic Year 2026–27

Department of Information Technology

A D Patel Institute of Technology

New Vallabh Vidyanagar, Vitthal Udyognagar – 388121

CERTIFICATE

This is to certify that the mini project work embodied in this report entitled "To-Do List Application using Python and Tkinter (TaskMate)" was carried out by Megh Chetankumar Patel (12402080601081) at A D Patel Institute of Technology for partial fulfilment of the requirements of the Bachelor of Technology degree in Information Technology awarded by Charutar Vidya Mandal (CVM) University. This project work has been carried out under the guidance and supervision of the faculty guide and is to the satisfaction of the department.

Project Evaluation Summary

Project Title: To-Do List Application (TaskMate GUI Application)

Core Concepts Verified: Python Lists, Functions, File Handling, and Tkinter GUI

Date: _____

Place: New Vallabh Vidyanagar

Lab Teacher / Faculty Guide	Head of Department
Keyur Patel Assistant Professor	Dr. Priyang Bhatt Associate Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to A D Patel Institute of Technology and the Department of Information Technology for providing the academic environment, computing laboratory facilities, and resources to undertake this mini project entitled “To-Do List Application using Python and Tkinter.”

I am thankful to my faculty guide and laboratory teacher, Keyur Patel, Assistant Professor, for guidance, suggestions, and encouragement throughout the design and development of this project. The project helped me understand how Python lists and functions can be used to manage tasks, how file handling can preserve data, and how Tkinter can provide a graphical user interface.

I would also like to thank the Head of the Department of Information Technology for supporting practical learning activities. I am grateful to my classmates and family for their support and encouragement during the completion of this mini project.

Megh Chetankumar Patel
(12402080601081)

Student Declaration & Repository Details

Student Name: Megh Chetankumar Patel

Enrollment Number: 12402080601081

Project: To-Do List Application using Python and Tkinter

Version Control: The project can be hosted on GitHub with its Python source files and documentation.

TABLE OF CONTENTS

Section	Page
CERTIFICATE	2
ACKNOWLEDGEMENT	3
TABLE OF CONTENTS	4
LIST OF TABLES AND FIGURES	5
ABSTRACT	6
CHAPTER 1: INTRODUCTION	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Objectives	7
1.4 Scope	8
1.5 Existing and Proposed System	8
1.6 Applications	8
CHAPTER 2: THEORETICAL BACKGROUND	9
2.1 Python Lists	9
2.2 Functions	9
2.3 File Handling	9
2.4 Tkinter GUI	9
2.5 Basic Complexity	10
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY	11
3.1 System Overview	11
3.2 Working of the System	11
3.3 Algorithm Design	11
3.4 Methodology and Data Flow	12
CHAPTER 4: IMPLEMENTATION AND TESTING	13
4.1 Technologies Used	13
4.2 Implementation	13
4.3 User Interface	13
4.4 Testing	13
4.5 Result and Discussion	14
CHAPTER 5: FUTURE SCOPE AND CONCLUSION	15
5.1 Future Scope	15
5.2 Conclusion	15
CHAPTER 6: REFERENCES	16

LIST OF TABLES AND FIGURES

List of Tables

Table 1: Comparison between manual task tracking and the proposed TaskMate GUI system

Table 2: Technologies and modules used in the project

Table 3: Test cases for the To-Do List Application

List of Figures

Figure 1: Three-layer system architecture of TaskMate

Figure 2: Data flow diagram of TaskMate

The architecture and descriptions in this report are based on the proposed simple To-Do List application using Python lists, functions, text-file handling, and Tkinter.

ABSTRACT

The To-Do List Application (TaskMate) is a desktop mini project developed using Python and Tkinter. It demonstrates four basic programming concepts: Lists, Functions, File Handling, and Graphical User Interface (GUI) design. The main objective is to provide a simple application where users can add tasks, view saved tasks, and delete tasks through buttons and a task list.

The application stores tasks in a Python list while it is running. Functions are used to keep the program organized and to separate operations such as adding, displaying, deleting, loading, and saving tasks. File handling is used to store the task list in a text file named tasks.txt, so tasks can be loaded again when the application is opened later.

The graphical interface is created with Tkinter, Python's standard GUI library. It contains an entry field for new tasks, a list box to display tasks, and buttons for adding and deleting tasks. The design is intentionally kept simple so that it can be understood, maintained, and extended by a beginner.

Key Technical Highlights

- Language and GUI: Python 3 and Tkinter.
- Data storage: Python list.
- Persistence: Text-file read and write operations using open().
- User actions: Add task, view tasks, and delete selected tasks.
- Code organization: Separate files for GUI, task functions, and file handling.

CHAPTER 1: INTRODUCTION

1.1 Background

People often keep daily activities in notebooks, on paper, or in memory. These methods can make it difficult to maintain an updated list of pending work. A simple computer application can make task management easier by displaying tasks in one place and allowing users to add or remove items when needed.

Python is suitable for a beginner-level task manager because it provides lists for storing multiple tasks, functions for organizing operations, file handling for saving information, and Tkinter for creating a graphical interface.

1.2 Problem Statement

The problem addressed by this project is to create a simple desktop application that lets a user manage a personal task list. The application should allow users to add tasks, view all current tasks, delete a selected task, and keep the task list available after the program is closed.

1.3 Objectives

- To create a simple and understandable To-Do List application using Python.
- To use a list to store and manage tasks.
- To use functions for adding, viewing, and deleting tasks.
- To save tasks to a text file and load them when the program starts.
- To create a graphical user interface using Tkinter.
- To provide basic feedback when a task is missing or an invalid selection is made.

1.4 Scope

The project covers basic personal task management on a local computer. It is intended for individual use and stores task information in a local text file. It does not require a database or internet connection. The simple design makes it suitable for demonstrating introductory Python programming concepts.

1.5 Existing and Proposed System

Existing System: A user may record tasks in a notebook or use a basic console program. Paper lists can be lost or become difficult to update, while console programs require typed commands.

Proposed System: TaskMate provides a small Tkinter window with a task entry field, a list of tasks, and action buttons. Tasks are held in a Python list and saved to tasks.txt whenever the list changes.

1.6 Applications

- Personal daily task planning.
- Tracking college assignments and practical work.
- Maintaining reminders for small activities.
- Learning the connection between Python data structures, functions, file handling, and GUI programming.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Python Lists

A list is a Python data structure that stores multiple items in an ordered collection. Lists can be changed after they are created. In this project, tasks = [] stores the task names. The append() method adds a task, while pop() removes a task by its position. A loop is used to display each task in the GUI.

2.2 Functions

A function is a named block of code that performs a particular job. Functions make a program easier to read and reuse. The project separates operations into functions such as add_task(), delete_task(), load_tasks(), save_tasks(), and start_app(). The GUI buttons call the appropriate functions when clicked.

2.3 File Handling

File handling allows a program to read information from a file and write information to it. TaskMate uses the open() function with a with statement to access tasks.txt. When the application starts, load_tasks() reads saved task lines. After a task is added or deleted, save_tasks() writes the updated list to the file. If the file does not exist yet, the program starts with an empty list.

2.4 Tkinter GUI

Tkinter is a standard Python library for building desktop graphical user interfaces. The project uses a Tk() window, Label, Entry, Listbox, and Button widgets. The mainloop() method keeps the window open and waits for user actions. Button commands connect the interface to the task functions.

2.5 Basic Complexity

Adding a task to the end of a Python list is generally an efficient operation. Displaying all tasks takes time proportional to the number of tasks. Removing a task from the list may require shifting later items. Saving the list rewrites the task file, so the time needed depends on the number of saved tasks.

2.6 Comparison Between Manual and TaskMate System

Feature	Manual / Basic Console Method	Proposed TaskMate GUI
Interface	Paper list or typed commands	Window with buttons and task list
Task storage	Notebook or temporary variables	Python list
Saving tasks	Manual writing or no saving	Automatic saving to tasks.txt
Deleting tasks	Erase or edit manually	Select a task and click delete
Ease of use	Requires manual updates	Tasks managed through GUI controls

Table 1: Comparison between manual task tracking and the proposed TaskMate GUI system

CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY

3.1 System Overview

TaskMate is organized into three simple layers. The presentation layer is the Tkinter interface that accepts user actions. The task management layer uses Python lists and functions to add, display, and delete tasks. The storage layer reads and writes task data in tasks.txt.

Layer	Components	Purpose
1. Presentation Layer	Tkinter window, entry field, list box, buttons	Accepts user input and displays tasks
2. Task Management Layer	Python list and task functions	Adds, displays, and removes tasks
3. Storage Layer	File handling and tasks.txt	Saves tasks and loads them on startup

Figure 1: Three-layer system architecture of TaskMate

3.2 Working of the System

When the program starts, it reads any existing tasks from tasks.txt. The GUI then displays the tasks in a list box. When the user enters a task and clicks Add Task, the task is added to the list and saved to the file. When the user selects a task and clicks Delete Selected Task, that task is removed from the list and the file is updated. The list box is refreshed after each operation.

3.3 Algorithm Design

Add Task Algorithm

1. Read the task from the entry field.
2. Remove extra spaces from the start and end.
3. Check that the task is not empty.
4. Add the task to the list.
5. Save the updated list to tasks.txt.
6. Refresh the task list in the GUI.

View Tasks Algorithm

7. Clear the visible list box.
8. Read each task from the Python list.
9. Insert each task into the list box.
10. If there are no tasks, keep the list empty and show a message when needed.

Delete Task Algorithm

11. Check whether the user has selected a task.
12. Find the selected task position.
13. Remove that task from the Python list.
14. Save the updated list to tasks.txt.
15. Refresh the list box.

3.4 Methodology

The project was developed in small steps. First, the list and task functions were prepared. Next, file handling was added to load and save tasks. After that, Tkinter widgets were used to create the interface. Finally, basic tests were planned for adding, displaying, deleting, and saving tasks.

3.5 Data Flow

Step	Input / Operation	Output
1	User enters a task	Task text is checked
2	Add function updates list	Task appears in the list box
3	File handling saves list	Updated tasks.txt
4	User selects and deletes a task	Task is removed
5	GUI refreshes	Current task list is displayed

Figure 2: Data flow of TaskMate

CHAPTER 4: IMPLEMENTATION AND TESTING

4.1 Technologies Used

Technology / Concept	Purpose in the Project
Python 3	Main programming language
Python List	Stores the task names in memory
Functions	Separates task operations into reusable blocks
File Handling	Reads and writes task data in tasks.txt
Tkinter	Creates the desktop window and controls
VS Code or Python IDLE	Can be used to edit and run the program

Table 2: Technologies and concepts used in the project

4.2 Implementation

The application is divided into small Python files to keep the code understandable. main.py starts the application. gui.py creates the Tkinter window and connects buttons to functions. task_functions.py contains operations for adding and deleting tasks. file_handling.py reads and writes tasks.txt. The task file is created when the program first saves tasks.

4.3 User Interface

The main window displays the title To-Do List Application, an entry field for typing a task, a list box containing saved tasks, and buttons to add or delete tasks. The user selects a task from the list before deleting it. The interface is intentionally simple so that the main functions are easy to find.

4.4 Testing

Test Case	Input / User Action	Expected Result
TC-1	Start the application when tasks.txt does not exist	Application opens with an empty task list
TC-2	Enter a task and click Add Task	Task appears in the list and is saved
TC-3	Click Add Task with an empty entry field	Error message asks the user to enter a task
TC-4	Select a task and click Delete Selected Task	Selected task is removed and changes are saved
TC-5	Close and reopen the application	Previously saved tasks are loaded
TC-6	Click delete without selecting a task	Error message asks the user to select a task

Table 3: Test cases for the To-Do List Application

4.5 Result and Discussion

The proposed design supports the required basic task-management operations: adding tasks, viewing tasks, deleting selected tasks, and saving tasks in a file. The list stores the current data, functions organize the program logic, file handling preserves the data between sessions, and Tkinter provides a graphical way to interact with the program. Actual test results should be filled in after running the program on the target computer.

CHAPTER 5: FUTURE SCOPE AND CONCLUSION

5.1 Future Scope

The current project is intentionally small and can be improved in several ways. A completion checkbox could allow users to mark tasks as finished. Due dates and task priorities could help users organize work. Search and category options could be added when the number of tasks becomes larger. The text file could later be replaced with SQLite if more structured storage is required. A reminder feature could also be introduced in a future version.

5.2 Conclusion

The To-Do List Application using Python and Tkinter demonstrates the practical use of Lists, Functions, File Handling, and GUI development. Python lists hold the tasks, functions separate the main operations, file handling saves and loads the task list, and Tkinter provides a graphical interface. The project is suitable for learning how different basic Python concepts work together to solve a simple everyday problem.

CHAPTER 6: REFERENCES

16. Python Software Foundation. “Data Structures — Lists.” Python 3 Documentation. https://docs.python.org/3/tutorial/datastructures.html
17. Python Software Foundation. “Defining Functions.” Python 3 Documentation. https://docs.python.org/3/tutorial/controlflow.html#defining-functions
18. Python Software Foundation. “Reading and Writing Files.” Python 3 Documentation. https://docs.python.org/3/tutorial/inputoutput.html#reading-and-writing-files
19. Python Software Foundation. “tkinter — Python Interface to Tcl/Tk.” Python 3 Documentation. https://docs.python.org/3/library/tkinter.html
20. Python Software Foundation. “Built-in Functions: open().” Python 3 Documentation. https://docs.python.org/3/library/functions.html#open